

使用神经网络进行昆虫分类

时分 101 PB23151799

2025 年 4 月 10 日

摘要

本实验为解决昆虫多分类问题，在 Pytorch 框架下设计了一个经典的多层全连接前馈神经网络（Feedforward Neural Network, FNN），在两组数据集上进行了实验，在测试集上分类准确率均接近 90%。接着，实验测试了参数个数、隐藏层个数和激活函数的选取对模型分类效果的影响，并简单分析了造成影响的原因。为解决第二组数据集中存在的噪声问题，本实验对比了三种去噪方式：人工去噪、基于 KMeans 聚类算法的去噪和基于模型预测的去噪，分别分析了其优点与劣势，并结合具体数据探讨了不同算法在本数据集上的表现。

一、前言

分类问题是数据分析领域与人工智能领域的重要基础任务。传统分类方法依赖人工特征提取，存在效率低、泛化性差等问题。随着深度学习技术的发展，基于神经网络的多分类方法为自动化昆虫识别提供了新思路。^[1]

本实验以昆虫多分类问题为背景，在 Pytorch 框架下，设计了多层全连接前馈神经网络（FNN），探究网络结构参数与去噪方法对分类性能的影响。通过两组数据集验证模型有效性，测试集准确率均接近 90%，并系统分析隐藏层数量、激活函数选择及噪声处理策略的优化方向。

同时，为解决第二组数据集中出现的噪声问题，本实验提出了三种去噪方式：人工去噪、基于 KMeans 聚类算法的去噪和基于模型预测的去噪（使用 Cleanlab 库），展示了不同去噪方式在第二组数据集上的表现，探索不同去噪力度对结果的影响。同时综合三种方法的效果，分别分析了方法的优点和劣势。

二、相关工作

目前，应用机器学习的分类问题主要采用两类方法：

- 传统机器学习方法：如支持向量机（SVM）和随机森林，依赖专家经验进行特征工程；
- 深度学习方法：卷积神经网络（CNN）在图像分类中表现优异，但全连接网络在结构化数据中仍具竞争力。

在噪声处理方面，现有研究多集中于图像去噪或基于统计的离群点检测，而针对分类任务的后验概率去噪方法研究较少。本实验系统分析了 FNN 超参数对分类性能的影响规律，并对比了传统人工去噪、聚类去噪与模型驱动去噪的效果。

三、问题分析

本实验需解决以下核心问题：

- 多分类任务建模：基于昆虫的体长、翼长两个特征，设计具有足够表达能力的网络结构；
- 模型结构影响：参数规模与隐藏层深度可能导致过拟合或欠拟合，需通过实验确定最优配置；
- 激活函数选择：不同非线性函数的梯度特性可能影响模型收敛速度与分类边界形态。
- 噪声干扰问题：第二组数据集包含标注噪声，需评估不同去噪方法对模型鲁棒性的提升效果；

四、建模的假设

- 输入数据特征与类别标签间存在可学习的非线性映射关系；

- 隐藏层神经元可通过组合非线性激活函数逼近真实分类边界；
- 数据集噪声主要表现为标注错误，而非传感器采集噪声；
- 测试集与训练集服从相同独立分布 (i.i.d. 假设)；
- 采用交叉熵损失函数可有效度量多分类预测误差。

五、符号说明

表 1: 符号说明

符号	说明	单位
$\mathbf{z}$	神经网络单层的向量	/
$\mathbf{W}$	传播权重	/
$\mathbf{b}$	传播误差	/
$\mathcal{L}$	损失函数	/

六、数学模型建立

6.1 神经网络结构定义

设输入层维度为 d ，第 k 个隐藏层含 n_k 个神经元，输出层维度为类别数 C 。对于第 l 层前向传播：

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$

其中 $\sigma(\cdot)$ 为激活函数，本文中出现的激活函数有：

- ReLU:

$$f(x) = \max(0, x)$$

- LeakyReLU:

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{otherwise} \end{cases} \quad (\alpha \in (0, 1), \text{通常取 } \alpha = 0.01)$$

- Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Softmax:

$$f(x) = \text{Softmax}(\mathbf{z}) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

6.2 损失函数

为解决多分类问题，损失函数选择经典且有效的交叉熵损失函数：

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(p_{ic})$$

6.3 优化方法

采用 Adam 优化器更新参数 $\{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}$ ，学习率可变，本实验中取 0.001。

七、结果（与对比）

7.1 FNN 的基础模型

本多分类问题有两个输入：昆虫的体长和翼长（浮点型），一个输出：昆虫的分类（取 0、1、2）。两组数据集的规模如表 2：

组	训练集大小	测试集大小（选自训练集）	测试集大小（新数据）
1	450	60	150
2	450	60	150

表 2: 两组数据集的规模

将数据集的内容进行可视化，如图 1 所示，按顺序为：训练集 1、测试集 1（来自训练集，下记为 1-a）、测试集 1（新数据，下记为 1-b）、训练集 2、测试集 2（来自训练集，下记为 2-a）、测试集 2（新数据，下记为 2-b）。

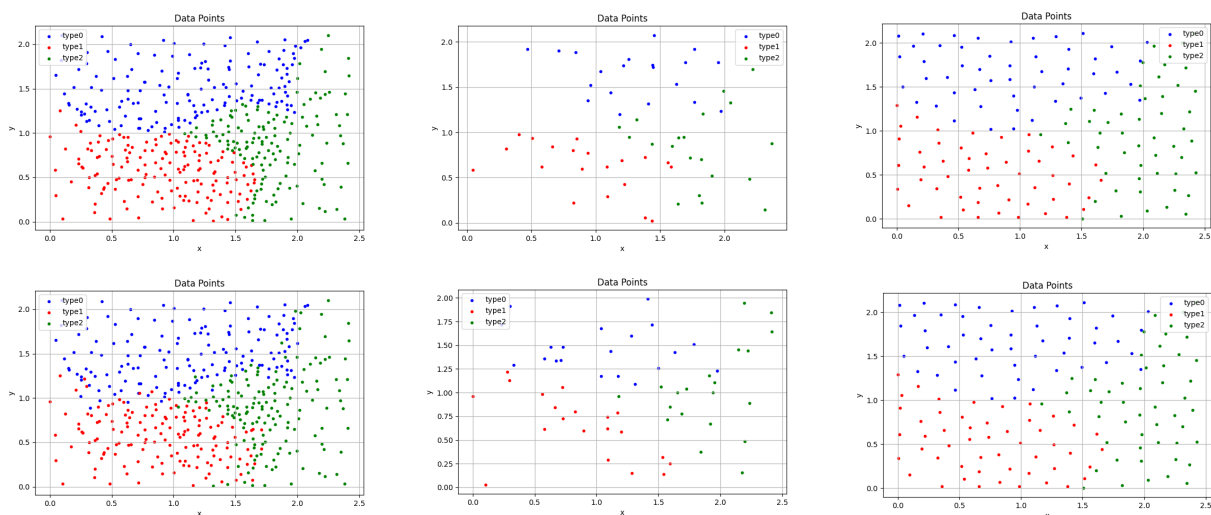


图 1: 数据集可视化

可以观察到数据集具有以下特点：

- 不同类数据呈现出明显的群聚现象；
- 不同类交界处数据较密集，在数值上下界处数据较稀疏；
- 第 2 组与第 1 组的训练集、测试集 b 具有相同的体长、翼长数据，但在类型分界处附近有一些数据的标签不同（即第 2 组数据的噪声）。

由于问题的规模较小，根据经验，模型实现准确分类需要的参数和层数一般不多；简单模型出现过拟合现象不明显，出于简化模型的考虑，在本实验中可以不划分验证集，也不选取随机 Dropout、添加正则化项等防止过拟合的方案。本实验的基础模型（对照组）使用 1 个隐藏层、64 个参数，激活函数为 ReLU，训练 100 轮，在两组数据集上进行测试（代码为 InsectClassifier.py）。测试以损失（Loss）、分类准确率（Accuracy）和测试集上的 ROC 曲线（ovr 模式）下面积（AUC）作为分类效果的衡量标准，结果如表 3 所示。

数据集组	1	2
训练集损失	0.2483	0.3178
测试集 a 损失	0.2538	0.2969
测试集 b 损失	0.2672	0.2384
训练集分类准确率	89.56%	86.00%
测试集 a 分类准确率	90.00%	88.33%
测试集 b 分类准确率	89.33%	89.33%
测试集 a 的 AUC	97.79%	97.46%
测试集 b 的 AUC	98.43%	98.57%

表 3: 基础模型在两组数据集上的最终表现

基础模型在两组数据集的训练集、测试集上均表现出接近 90% 的分类准确率，在测试集上的 AUC 均在 98% 左右，这体现了 FNN 模型在处理多分类问题中的有效性。同时，测试集 a、b 上的损失、分类准确率大致相当，这说明模型在训练中没有出现明显的过拟合现象。

为直观的表现出训练轮数是否足够、参数是否收敛，本实验可视化了训练过程中的损失和分类准确率变化曲线，如图 2 所示。

图中的损失曲线与分类准确率曲线均趋于稳定，这说明训练轮数设为 100 对该模型已经足够。表示训练集、测试集 a、测试集 b 的 3 条曲线的趋势大致保持相同，变化速度由快减慢。同时，注意到在处理数据集 2 的后期，曲线表现出较强烈的震荡，猜想这可能是由数据集 2 中存在噪声导致的，这一猜想将在报告后文中与去噪后的结果对比中进行验证。

为了更直观地展示分类效果，本实验以散点图形式展示两个数据集上最终的分类结果，黑点表示分类正确、红点表示分类错误，如图 3 所示。

可以发现，模型在分类中所犯的错误集中在三种昆虫类型的交界处，这说明模型的关键在于划定分类的界限（在后文训练集 1、2 的噪声对比中可以发现噪声也集中在这些位置，这说明人工分类任务也以处理这一问题作为关键）；模型在处理极大、极小值的分类时表现良好，表现出接近 100% 的分类准确率，这一点与后文中提到的 KMeans 聚类方法进行分类所表现出的特征有明显区别。

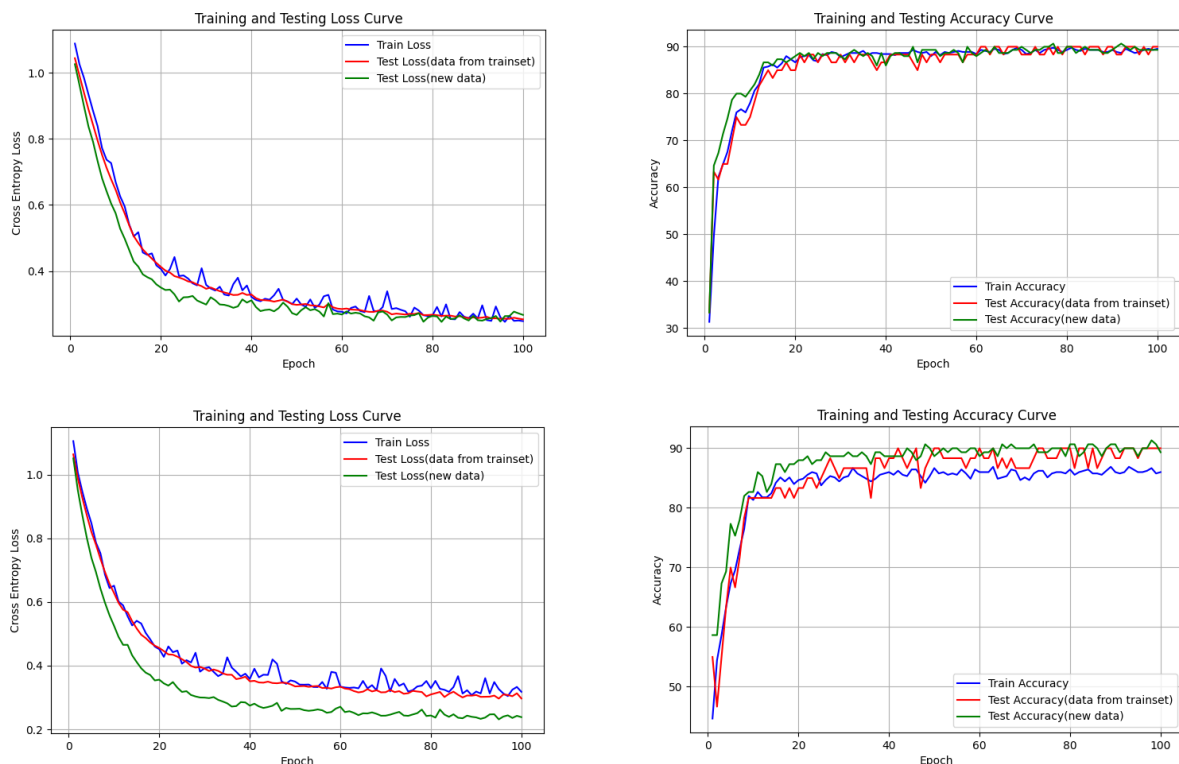


图 2: 基础模型在两组数据集上的损失、分类准确率变化曲线

7.2 神经网络结构对模型分类效果的影响

7.2.1 隐藏层参数个数的影响

隐藏层参数的个数一般对分类器的效果有明显影响：个数越少，分类精度越低，可能出现未拟合现象；个数越多，分类精度越高，但更可能出现过拟合问题。为探索隐藏层参数的个数对模型分类效果的影响，本实验在基础模型上调整得到了 5 个实验组，隐藏层参数分别为 128、256、512、1024、2048 个，各项分类结果指标如表 4、图 4所示。

隐藏层参数个数	64	128	256	512	1024	2048
训练集损失	0.2483	0.2342	0.1947	0.2501	0.1437	0.1457
测试集 a 损失	0.2538	0.2355	0.1998	0.1592	0.1456	0.1955
测试集 b 损失	0.2672	0.2492	0.2181	0.1864	0.1516	0.1190
训练集分类准确率	89.56%	89.11%	91.78%	94.44%	93.78%	94.89%
测试集 a 分类准确率	90.00%	86.67%	91.67%	93.33%	98.33%	93.33%
测试集 b 分类准确率	89.33%	89.33%	90.67%	93.33%	94.67%	95.33%
测试集 a 的 AUC	97.79%	98.00%	98.62%	99.00%	99.04%	99.38%
测试集 b 的 AUC	98.43%	98.60%	98.87%	99.25%	99.57%	99.71%

表 4: 不同隐藏层参数个数对分类结果的影响

随着隐藏层参数增加，测试集损失呈现下降趋势、分类准确率、AUC 呈现上升趋势，体现了分类精

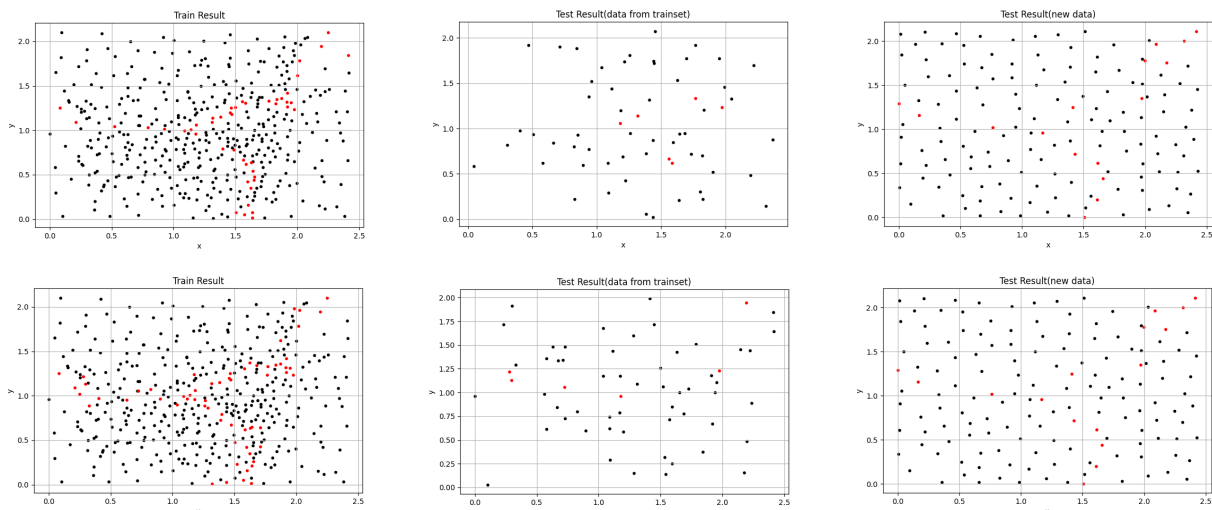


图 3: 基础模型在两组数据集上的分类结果展示

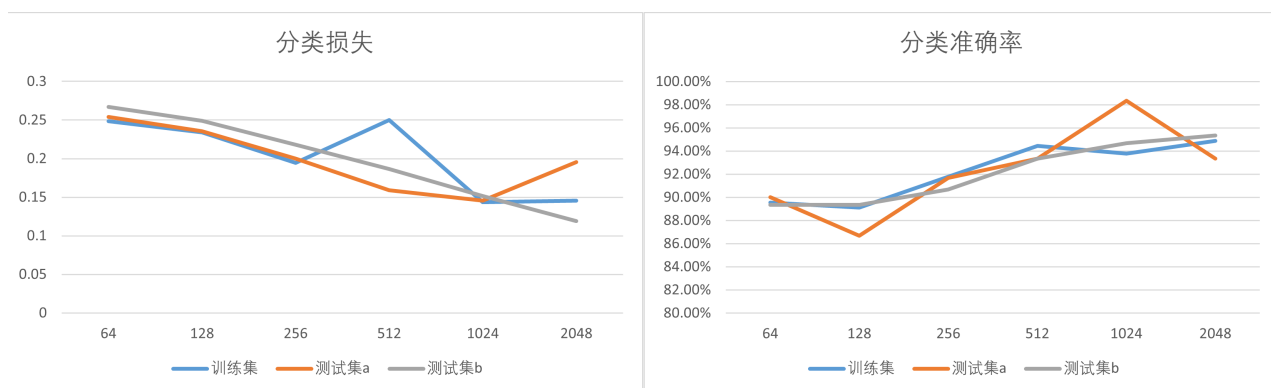


图 4: 不同隐藏层参数个数的损失、分类准确度变化

度的提升；但在 1024 组，测试集 a 上的损失和分类准确率相对测试集 b 出现了明显优势，这表明模型出现了过拟合现象。实验结果与上述理论保持一致。

同时，对 2048 组的损失和分类准确度曲线进行单独分析，如图 5 所示。

发现损失曲线和分类准确度曲线出现了极剧烈的震荡，这表明处理简单问题的隐藏参数过多时，模型的稳定度会有所下降。

综合各组的表现，认为对于单隐藏层，选取 128 256 个参数是较优的：既具有较高的分类准确度，也避免了出现明显的过拟合问题。

7.2.2 隐藏层层数的影响

隐藏层的层数对分类器效果的影响与隐藏层参数个数的影响有类似之处：层数越少，分类精度越低，可能出现未拟合现象；层数越多，分类精度越高，但更可能出现过拟合问题。为探索隐藏层参数的个数对模型分类效果的影响，本实验在基础模型上调整得到了 4 个实验组，隐藏层层数分别为 2 (64+32)、3 (128+64+32)、4 (128+64+32+10)、5 (128+64+32+32+10) 个，各项分类结果指标如表 5 所示。

与隐藏层参数个数类似，在 1 3 层范围随着隐藏层参数增加，测试集损失呈现下降趋势、分类准确率、AUC 呈现上升趋势，分类精度提升，但这一趋势在 3 5 层体现不明显，说明多于 3 层的 FNN 在此

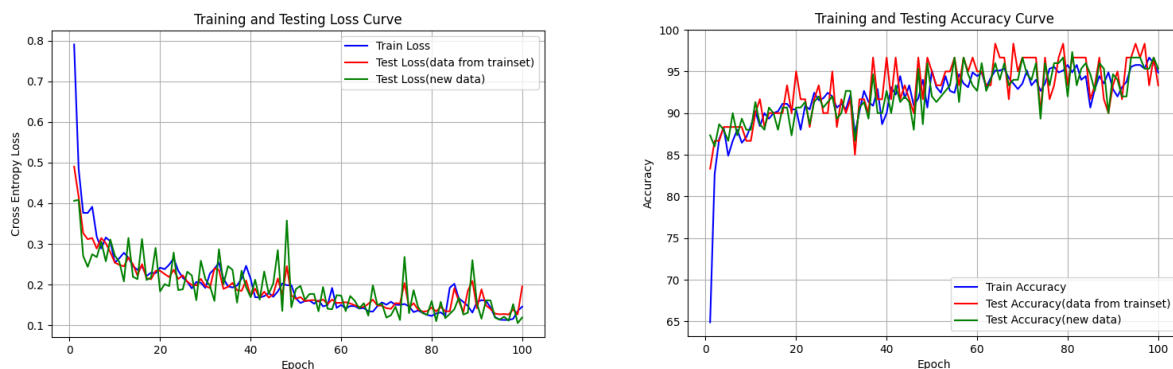


图 5: 有 2048 个隐藏层参数的损失、分类准确度曲线

隐藏层层数	1	2	3	4	5
训练集损失	0.2483	0.2260	0.1035	0.1116	0.1316
测试集 a 损失	0.2538	0.1658	0.0957	0.0953	0.1022
测试集 b 损失	0.2672	0.2092	0.1082	0.1163	0.1595
训练集分类准确率	89.56%	92.22%	95.78%	95.56%	94.44%
测试集 a 分类准确率	90.00%	93.33%	98.33%	98.33%	98.33%
测试集 b 分类准确率	89.33%	92.00%	98.00%	96.00%	92.67%
测试集 a 的 AUC	97.79%	99.00%	99.62%	99.46%	99.50%
测试集 b 的 AUC	98.43%	99.00%	99.73%	99.73%	99.51%

表 5: 不同隐藏层层数对分类结果的影响

分类任务中并不具有优越性；同时，在 4、5 层，测试集 a 上的损失和分类准确率相对测试集 b 出现了明显优势，这表明模型出现了过拟合现象。

和上文类似地，对 5 层组的损失和分类准确度曲线进行单独分析，如图 6 所示。

该组的损失、分类准确度曲线也出现了明显的震荡，同样展现出应对简单问题时多层 FNN 的不稳定特征。

综合各组的表现，认为设置 2 3 个隐藏层是较优的，即在具有高分类准确度的同时，也不具有明显的过拟合问题。设置 3 个隐藏层时，在测试集 a、b 上的分类准确率均达到 98%，代表了本实验中模型的最理想情况。

7.2.3 激活函数的影响

在 FNN 中，不同的激活函数具有不同的性质和优缺点；应用不同激活函数的参数收敛速度也有差异。本实验在基础模型上调整得到了 4 个实验组，分别采用 LeakyReLU、Tanh、Sigmoid、Softmax 作为激活函数，各项分类结果指标如表 6 所示。

同时分析不同激活函数下损失曲线，以估计参数的收敛速度，如图 7 所示，激活函数依次为 ReLU、LeakyReLU、Tanh、Sigmoid、Softmax。

从数据上看，ReLU、LeakyReLU、tanh 均表现出良好的性能，Sigmoid 和 Softmax 的性能则较差；从损失曲线上分析，发现前三者损失下降速度均较快，且呈现出先快后慢的趋势，这表明在前三个损失

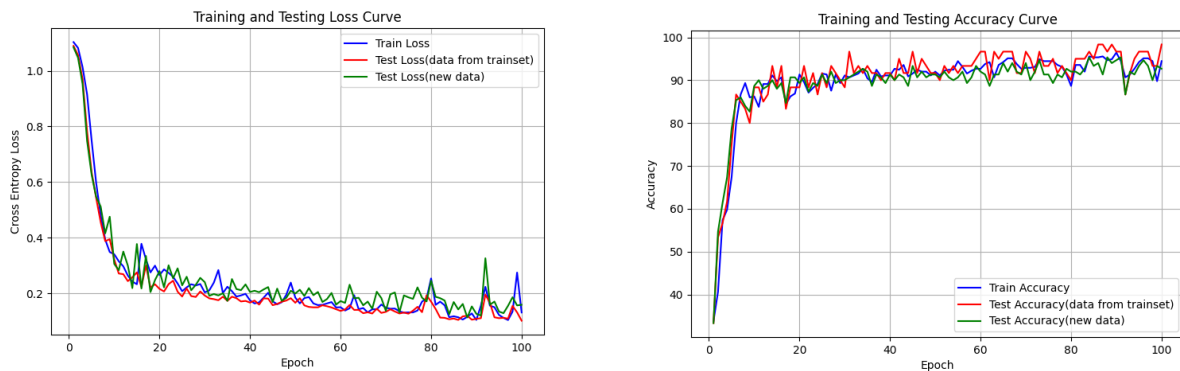


图 6: 有 5 个隐藏层的损失、分类准确度曲线

激活函数	ReLU	LeakyReLU	tanh	Sigmoid	Softmax
训练集损失	0.2483	0.2932	0.2644	0.4016	0.6188
测试集 a 损失	0.2538	0.2648	0.2752	0.3741	0.5986
测试集 b 损失	0.2672	0.2632	0.2759	0.3216	0.5693
训练集分类准确率	89.56%	89.11%	88.00%	88.00%	88.44%
测试集 a 分类准确率	90.00%	90.00%	86.67%	88.33%	86.67%
测试集 b 分类准确率	89.33%	88.67%	88.67%	89.33%	85.33%
测试集 a 的 AUC	97.79%	97.79%	97.58%	97.58%	96.62%
测试集 b 的 AUC	98.43%	98.39%	98.29%	98.23%	96.60%

表 6: 不同激活函数对分类结果的影响

函数下参数得到了快速的收敛；后二者损失下降速度则明显较慢，乃至呈现线性下降的趋势，这表明选择后二者情况下的参数收敛性能较差。同时考虑时间复杂度，ReLU 和 LeakyReLU 得益于其线性表达式，在正、反向传播求梯度时显示出明显的时间优势，而后三者则在时间上略逊于前二者。

同时考虑各个损失函数的优缺点：

- ReLU 虽然形式简单，但会抛弃小于 0 的数值，这会导致部分神经元失去作用，称为“死亡神经元”现象。
- LeakyReLU 具有 ReLU 形式简单的优势，同时也通过引入负值部分的线性函数避免了“死亡神经元”现象，但负值部分的系数在处理困难问题时需要微调，且在面对大量数据时容易出现过拟合现象。
- tanh 和 Sigmoid 可以输出平滑的梯度，但在数值绝对值很大时，梯度会接近 0，即“梯度消失”现象，导致部分神经元失去作用。
- Softmax 适用于处理概率形式的数值，且输出也为归一化的概率形式，但不适用于处理大数值；在输入数值前往往需要进行归一化操作。

综合数据、损失曲线、时间复杂度和函数特性，认为 LeakyReLU 在处理本实验问题时不存在过拟合现象，且具有较低的时间复杂度、较高的分类准确率和较快的参数收敛速度，是较优的损失函数。

7.3 数据集的去噪算法

噪声是影响机器学习算法有效性的重要因素之一，由于实际数据集存在采集误差、主观标注以及被恶意投毒等许多因素，使得所构造的数据集中难免存在噪声。噪声可能会降低数据的利用率、使模型的性能下降、复杂度增加。^[2] 基于可视化分析数据，发现数据集 1 与数据集 2 的训练集中点的位置均相同，但存在 42 个不同的标签，即数据集 2 中存在标签噪声。

由于 FNN 本身具有一定的噪声过滤功能，基础模型在处理数据集 2 时仍表现出较高的分类准确率和较低的损失，但模型的损失曲线出现了较明显的震荡，即模型的稳定性有所下降。为了减少噪声数据的影响，提高模型的稳定性，需要引入数据集去噪操作。报告将介绍三种去噪方式：传统的人工去噪、基于 KMeans 聚类算法的去噪和基于模型预测的去噪（使用 Cleanlab 库）。

7.3.1 人工去噪

人工去噪是最经典的去噪方法，即通过人工的方式选取出噪声并加以过滤。如果负责去噪的人员（即专家）足够优秀，人工去噪可以具有很高的准确度，但不可避免地带来了高时间、经济成本问题，而低质量的人工去噪更是连准确度都无法保证。

由于数据集 1 的准确性，我们可以把数据集 1 的训练集视为人工去噪得到的训练集，其去除的噪声点如图 8 所示。可以看到，噪声点集中分布在不同类型的交界处，共有 42 个噪声点。

基于去噪后的数据集，利用基础模型进行训练，与去噪前的结果对比如表 7 所示。

是否去噪	去噪前	去噪后
训练集损失	0.3178	0.2569
测试集 a 损失	0.2969	0.2961
测试集 b 损失	0.2384	0.2531
训练集分类准确率	86.00%	89.33%
测试集 a 分类准确率	88.33%	90.00%
测试集 b 分类准确率	89.33%	90.00%
测试集 a 的 AUC	97.46%	97.54%
测试集 b 的 AUC	98.57%	98.49%

表 7: “人工去噪”前后的分类效果对比

同时分析二者的损失曲线，如图 9，左为去噪后曲线。

相较于去噪前，去噪后的分类准确率有所上升，损失和 AUC 有所下降，损失曲线的后半段震荡也更小。这表明在解决数据集 2 上的分类任务时，去噪操作有利于提升分类效果，同时提高模型的稳定性。

7.3.2 基于 KMeans 聚类算法的去噪

聚类是常见的无监督学习方法，简单地说就是把相似的数据样本分到一组（簇）。在聚类的过程中，我们并不清楚某一类是什么（通常无标签信息），需要实现的目标只是把相似的样本聚到一起，即只是利用样本数据本身的分布规律。^[3]

KMeans 聚类是一种最常见的划分聚类算法，其原理是先初始化 k 个簇类中心，基于计算样本与中

心点的距离归纳各簇类下的所属样本，迭代实现样本与其归属的簇类中心的距离为最小的目标，即：

$$\operatorname{argmin}_C J(C) = \sum_{k=1}^K \sum_{x^{(i)} \in \mathbb{C}_k} \|x^{(i)} - \mu^{(k)}\|_2^2$$

在本数据集中，利用 KMeans 聚类算法，可以将数据集划分为 3 个簇，对应 3 种昆虫类型；在聚类算法中，将与簇中心距离较远的数据点视为噪声，可以将其过滤，从而完成去噪操作。

通过调整距离计算的系数 (threshold_multiplier)，可以调整去噪的强度，即调整删除数据点的个数。在数据集 2 上应用 KMeans 聚类算法去噪，设置不同的去噪强度以探索最优设置，去噪效果如表 8 所示。

距离系数	0.8	1	1.2	1.4
去除点总数	89	70	53	42
实际噪声点个数	42	42	42	42
准确去除点个数	22	18	14	12
准确率	52.38%	42.86%	33.33%	28.57%

表 8: 应用 KMeans 聚类去噪的效果

距离系数越大，去除的噪声点总数越少，去除的准确率也会下降，噪声过滤效果较差；距离系数越小，去除的准确率上升，但去除的数据点总数较多，容易破坏数据的完整性。为探索最佳设置，将基础模型应用于经不同强度去噪的数据集，其分类结果如表 9 所示。

距离系数	未去噪	0.8	1	1.2	1.4
训练集损失	0.3178	0.2526	0.233	0.2425	0.2571
测试集 a 损失	0.2969	0.3543	0.3474	0.3669	0.3307
测试集 b 损失	0.2384	0.3029	0.2797	0.2868	0.2579
训练集分类准确率	86.00%	91.14%	90.79%	90.93%	89.95%
测试集 a 分类准确率	88.33%	86.67%	88.33%	86.67%	88.33%
测试集 b 分类准确率	89.33%	89.33%	89.33%	89.33%	89.33%
测试集 a 的 AUC	97.46%	96.75%	96.92%	96.62%	97.00%
测试集 b 的 AUC	98.57%	97.98%	98.21%	98.07%	98.45%

表 9: 应用 KMeans 聚类去噪前后的分类效果对比

为研究对模型稳定性的影响，取系数为 1.0 的训练损失曲线与未去噪的训练损失曲线作对比，如图 10 所示，左图为去噪后的效果。

应用 KMeans 去噪的损失曲线的震荡远小于去噪前的损失曲线，这说明模型在训练中呈现出更稳定的状态，去噪有效地提升了模型的稳定性。同时，去噪后模型在训练集上的表现明显优于去噪前，说明去噪对于提升模型分类效果也有一定帮助。但是，去噪前后模型在测试集上的表现并没有明显提升，为了分析该现象的原因，需要可视化去噪算法的结果。

应用 KMeans 聚类算法（距离系数为 1.0）去噪的结果可视化如图 11 所示。

注意到算法去除的可疑噪声很多都分布在极大、极小值处，这是因为这些位置的数据分布较稀疏，与簇中心的距离较远。然而实际上的噪声点大多分布在类的边界处，这导致去噪的效果有限，可能是导致

模型在训练集上表现没有明显提升的原因。为了更准确地找到噪声点，需要引入其他的去噪方法，比如基于模型预测的去噪方法。

7.3.3 基于模型预测的去噪

既然模型容易在噪声点上犯错误，那么找到模型预测的结果中置信度最低的点，就成为了一种可行的去噪方法。为了执行这种去噪方法的预训练过程称为置信学习。C.G.Northcutt 等^{[4][5]}详细研究了这种去噪方法，通过计算置信学习模型的预测概率与噪声出现概率的联合分布，自动预测可能的噪声点并加以清除。他们将与这一算法相关的诸多类和方法整合为 Cleanlab 库，其功能包括：

- 在数据集中查找问题（噪声）并按质量对数据点进行排名；
- 在有标签问题的任何数据集上训练任何分类器；
- 在数据集级别查找要合并和/或删除的重叠类；
- 衡量数据集的整体标签健康状况。

在本实验的去噪任务中，为对比实验考虑，仅使用“在数据集中查找问题（噪声）并按质量对数据点进行排名”的功能，置信学习的模型则选择本实验的基础模型。基于 Cleanlab 库的去噪结果（参数为 2.0）可视化如图 12。

与 KMeans 聚类方法去噪的结果不同，基于模型预测的去噪算法预测的噪声点均位于类的边界处，这与实际情况是吻合的。与 KMeans 类似，此算法也可以通过调整系数（frac_noise）调整去噪的强度。设置不同的去噪强度以探索最优设置，去噪效果如表 10 所示。

系数	1	1.5	2	2.5
去除点总数	26	40	46	50
实际噪声点个数	42	42	42	42
准确去除点个数	14	18	18	21
准确率	33.33%	42.86%	42.86%	50.00%

表 10: 应用模型预测去噪的效果

基于模型预测去噪的准确率明显优于 KMeans 聚类。与 KMeans 聚类去噪类似，系数越小，去除的准确率下降，噪声过滤效果较差；系数越大，去除的准确率上升，但去除的数据点总数较多，容易破坏数据的完整性。为探索最佳设置，将基础模型应用于经不同强度去噪的数据集，其分类结果如表 11 所示。

为研究对模型稳定性的影响，取系数为 2.0 的训练损失曲线与未去噪的训练损失曲线作对比，如图 13 所示，左图为去噪后的效果。

应用模型预测去噪的损失曲线的震荡同样远小于去噪前的损失曲线，这说明模型在训练中呈现出更稳定的状态，去噪有效地提升了模型的稳定性。同时，去噪后模型在训练集上的表现明显优于去噪前，在测试集上的表现不劣于去噪前，说明去噪对于提升模型分类效果也有一定帮助。

综合三种去噪方法，去噪操作均能有效提升模型在训练中的稳定性。对比来看，人工去噪虽然效果可能较好，但成本高、对专家专业性需求强；KMeans 聚类方法时间复杂度低，但缺乏对噪声问题的具体感知，噪声预测结果不够准确；基于模型预测的去噪虽然需要时间进行预训练，但噪声预测的准确性高、效果好、成本低，是解决数据集去噪问题的的一个较理想的算法。

系数	未去噪	1	1.5	2	2.5
训练集损失	0.3178	0.2146	0.1674	0.1656	0.1344
测试集 a 损失	0.2969	0.3520	0.3435	0.2970	0.3620
测试集 b 损失	0.2384	0.2798	0.2542	0.2398	0.2783
训练集分类准确率	86.00%	91.51%	94.15%	95.79%	96.25%
测试集 a 分类准确率	88.33%	86.67%	88.33%	88.33%	86.67%
测试集 b 分类准确率	89.33%	89.33%	89.33%	89.33%	89.33%
测试集 a 的 AUC	97.46%	96.96%	96.88%	90.00%	96.92%
测试集 b 的 AUC	98.57%	98.15%	98.37%	98.75%	98.29%

表 11: 应用模型预测去噪前后的分类效果对比

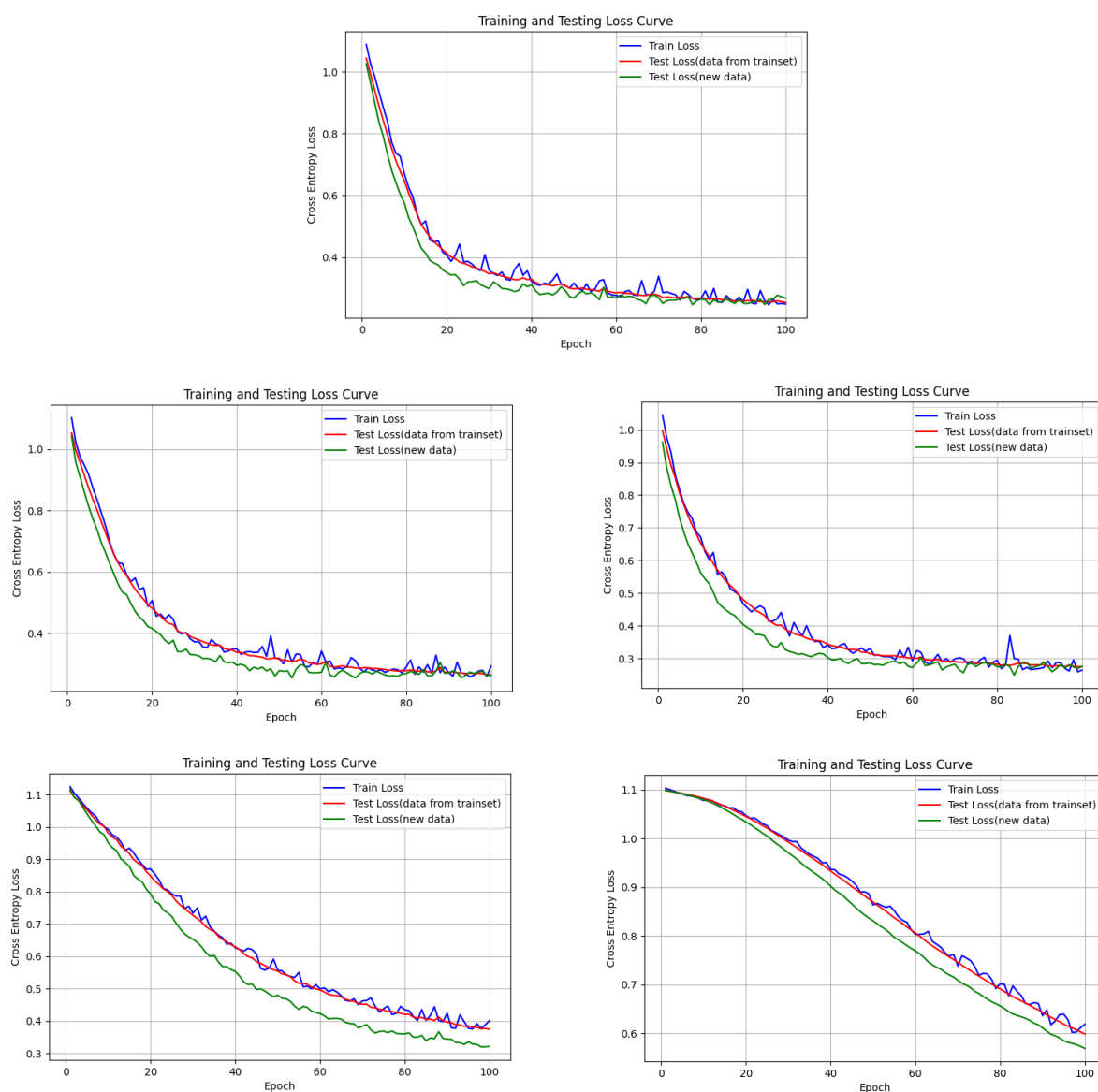


图 7: 不同激活函数的损失曲线

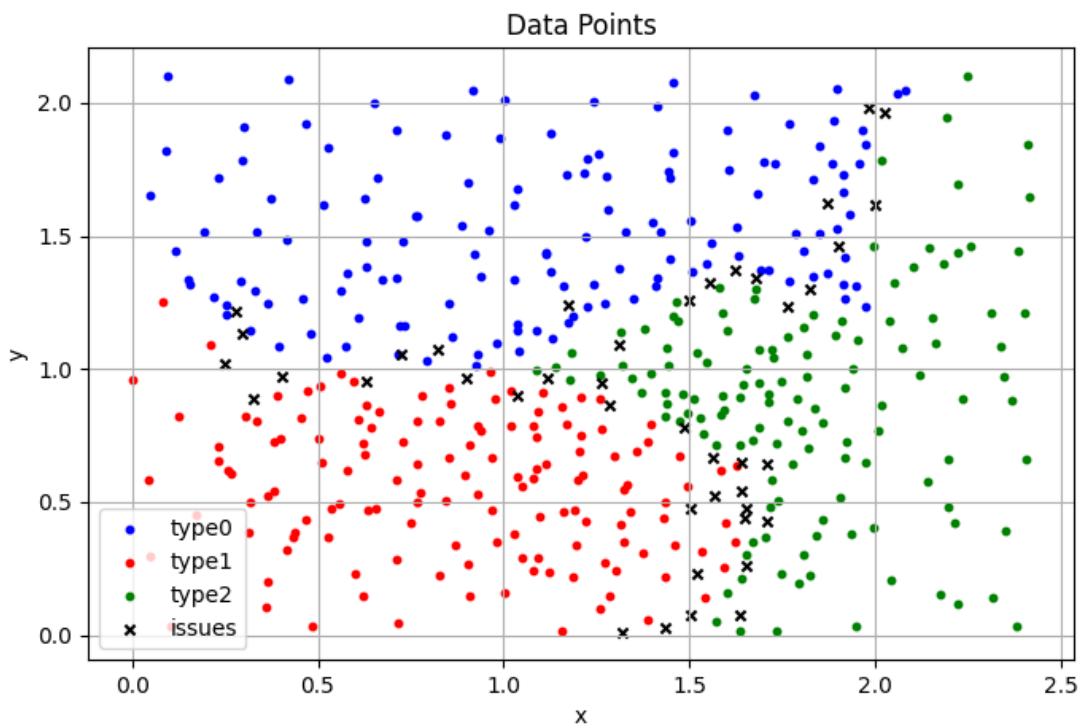


图 8: “人工去噪” 的可视化结果

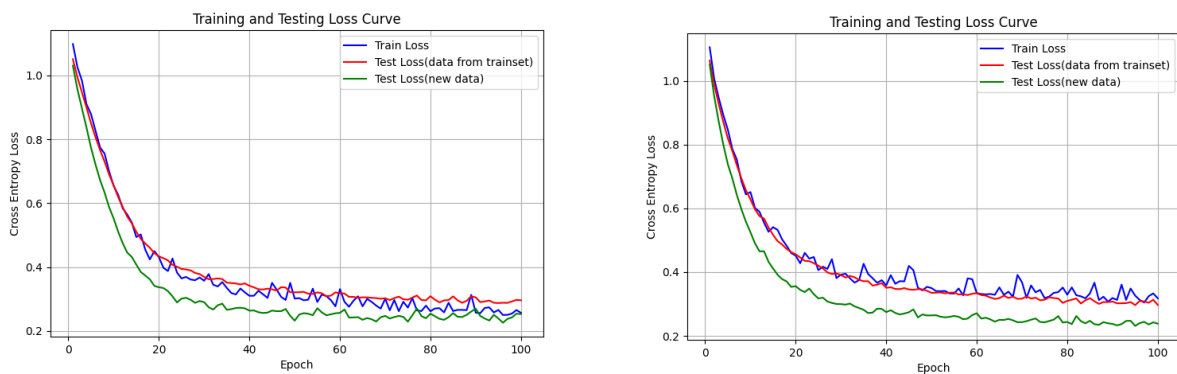


图 9: “人工去噪” 前后的损失曲线对比

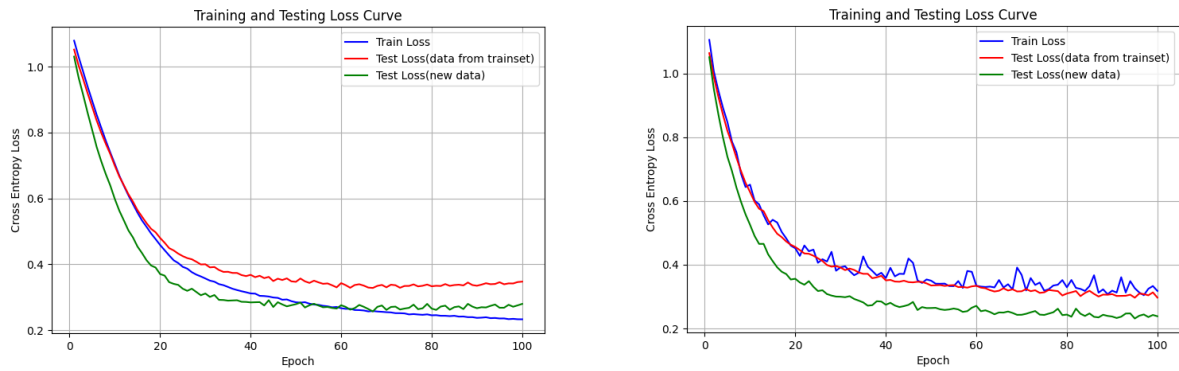


图 10: 应用 KMeans 聚类去噪前后的损失曲线对比

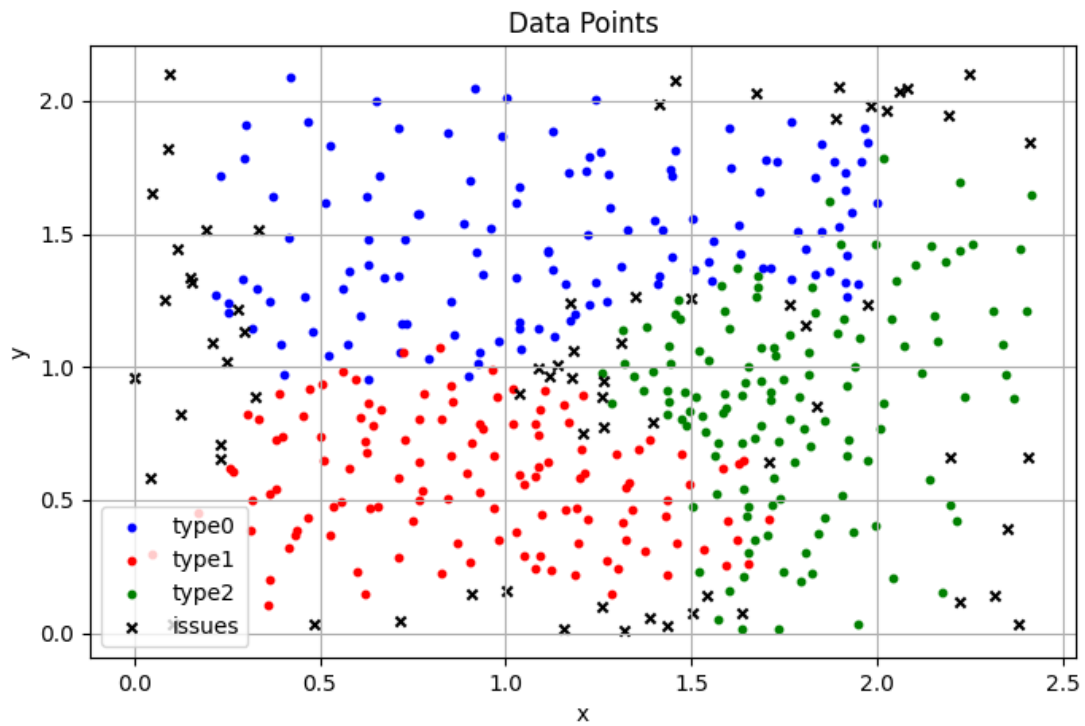


图 11: 应用 KMeans 聚类算法去噪的可视化结果

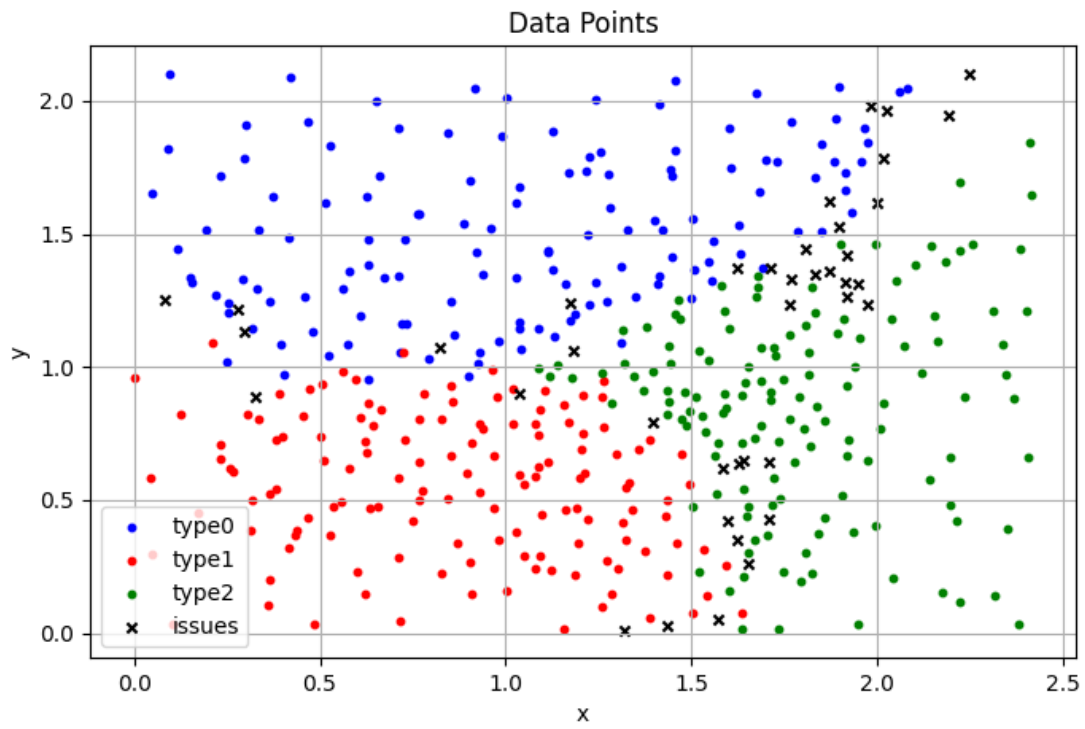


图 12: 应用模型预测去噪的可视化结果

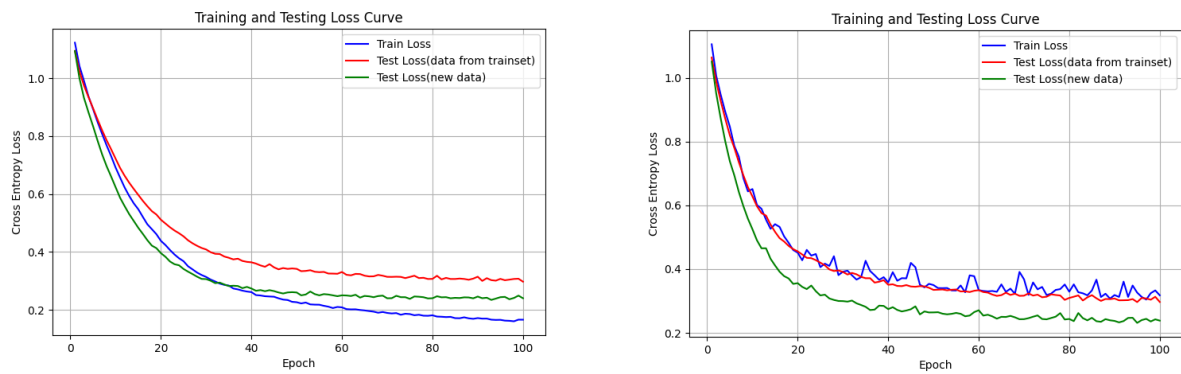


图 13: 应用模型预测去噪前后的损失曲线对比

八、结论

本实验为解决昆虫多分类问题，在 Pytorch 框架下设计了一个经典的多层全连接前馈神经网络，在两组数据集上进行了实验，在测试集上分类准确率均接近 90%。接着，实验测试了参数个数、隐藏层个数和激活函数的选取对模型分类效果的影响，简单分析了造成影响的原因，找到了相对最优的配置：128 256 个隐藏参数；隐藏层层数 2 3 层；选择 LeakyReLU 激活函数。为解决第二组数据集中存在的噪声问题，本实验对比了三种去噪方式：人工去噪、基于 KMeans 聚类算法的去噪和基于模型预测的去噪，分别分析了其优点与劣势，并结合具体数据探讨了不同算法在本数据集上的表现，认为基于模型预测的去噪算法可以相对最优地解决本问题。

九、问题

9.1 其他分类模型

本实验选择了最经典的 FNN 模型实现多分类任务。在分类任务中，还有许多其他模型可以利用，如逻辑回归、决策树、随机森林等。与这些分类模型的效果进行对比，可以更有力地证明 FNN 模型的有效性，或是找到更适合解决多分类问题的模型。

9.2 去噪算法中的问题

基于 KMeans 聚类的去噪算法中，簇的中心位置是随机选取后再调整的，这可能导致分类与聚类不完全匹配。如果能够分析数据整体的结构，选择合适的中心位置，可能可以进一步增强 KMeans 聚类算法的去噪效果。

基于模型预测的去噪算法中，分类模型选择了本实验的基础模型（FNN），可以考虑进一步探索该模型的神经网络结构、超参数配置对去噪效果的影响，也可以引入其他的分类模型，对比去噪效果等。

参考文献

- [1] SPSSPRO, “数学建模必备五大模型之一分类模型详解,” 2024.
- [2] HenrySmale, “人工智能安全-3-噪声数据处理,” 2023.
- [3] Twcat_tree, “【机器学习】kmeans 聚类算法,” 2023.
- [4] C. Northcutt, L. Jiang, and I. Chuang, “Confident learning: Estimating uncertainty in dataset labels,” *Journal of Artificial Intelligence Research*, vol. 70, pp. 1373–1411, 2021.
- [5] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive label errors in test sets destabilize machine learning benchmarks,” *arXiv preprint arXiv:2103.14749*, 2021.